

AtomCodec-Rerank

A Deployable Evidence-Reranking Architecture for Raw-Text Context Compression

Core thesis. Context compression should not call a large teacher LLM at inference time. The deployed system should score raw text chunks cheaply, select the minimum useful evidence under a token budget, and send ordinary text to the target LLM. ATOMCODEC-RERANK trains a cross-encoder evidence reranker from supporting-fact supervision, then optionally distills it into a cheap production selector.

Architecture Report

June 2026

Contents

1	Executive Summary	3
2	Problem Statement	3
2.1	Why Not Runtime LLM Compression?	3
3	High-Level Architecture	4
3.1	Main Components	4
4	Raw-Text Chunking	5
4.1	Chunking Rules	5
4.2	Chunking Failure Modes	5
5	Stage-1 Candidate Generation	5
5.1	Hybrid Candidate Score	5
6	Cross-Encoder Evidence Reranker	6
6.1	Model Form	6
6.2	Why This Is the Right Model Layer	6
7	Training Data Construction	6
7.1	Primary Dataset: HotpotQA	6
7.2	Secondary Dataset: QASPER	7
7.3	Hard Negative Mining	7
8	Training Objective	7
8.1	Binary Evidence Loss	7
8.2	Pairwise Ranking Loss	7
8.3	Risk-Weighted Loss	7
9	Budgeted Selection	8
9.1	Optimization Target	8
9.2	Greedy Approximation	8
10	RiskGuard: Cheap Add-Back Without LLM Verification	8
10.1	Risk Rules	8
11	Distilled Cheap Student	9
11.1	Teacher Scores	9
11.2	Student Features	9
11.3	Student Objective	9
12	Inference Modes	10
13	Computational Complexity	10

14 Evaluation Plan	10
14.1 Evidence Retrieval Metrics	10
14.2 Answer Quality Metrics	10
14.3 Compression Metrics	11
14.4 Critical-Loss Stress Tests	11
15 Baselines	11
16 Ablation Studies	12
17 Failure Modes and Mitigations	12
17.1 Stage-1 Retrieval Misses Gold Evidence	12
17.2 Chunk Boundary Corrupts Meaning	12
17.3 Cross-Encoder Overfits HotpotQA	12
17.4 Reranker Scores Are Not Token-Aware	12
17.5 RiskGuard Adds Too Much	12
17.6 Distilled Student Loses Semantics	12
18 Decision Junctions	13
19 Eight-Hour Build Plan	13
20 Implementation Skeleton	13
21 Startup Positioning	14
22 Conclusion	14

1 Executive Summary

ATOMCODEC-RERANK is a raw-text, extractive prompt compression system. It takes a long context and a query, divides the context into contiguous text chunks, scores the chunks by their likelihood of being answer-critical evidence, selects chunks under a hard token budget, and sends the selected raw text to the final LLM. The system is designed around one non-negotiable constraint:

No runtime teacher LLM. No runtime atom extraction. No runtime full-context verifier.

A large model may be used offline for labeling, auditing, or synthetic data generation, but it is not part of the deployment hot path. The main quality model is a query-conditioned cross-encoder evidence reranker, preferably initialized from a modern encoder-only backbone such as ModernBERT. The production story has two inference modes:

- **Quality mode:** BM25/dense retrieval produces top- K candidate chunks; a trained cross-encoder reranks them; a budget allocator selects the best raw chunks.
- **Cheap mode:** cross-encoder scores are distilled into a smaller MLP, LightGBM, or logistic model over cheap features; this approximates the reranker without transformer inference in the hot path.

The key product value is not generic summarization. It is query-conditioned evidence preservation under cost constraints. The model learns which chunks are likely to be necessary for answering, including boring but decisive spans containing exceptions, numbers, negations, function signatures, dates, policy constraints, or table rows.

2 Problem Statement

Let a prompt contain a context document C and query q . The context is tokenized into $T(C)$ tokens. A final LLM call using the entire context is expensive when $T(C)$ is large. We need a compressor g that returns a shorter raw-text context $\tilde{C}$ such that:

$$\tilde{C} = g(C, q, B), \tag{1}$$

$$T(\tilde{C}) \leq B, \tag{2}$$

$$\text{Ans}(q, \tilde{C}) \approx \text{Ans}(q, C), \tag{3}$$

where B is a token budget and Ans is the downstream answer produced by the final LLM.

The system is not a latent prompt compressor. It does not output hidden vectors or soft tokens. It outputs selected raw text spans. This matters because most hosted LLM APIs accept text, not arbitrary latent embeddings. Raw-text extractive compression is deployable, human-debuggable, and model-agnostic.

2.1 Why Not Runtime LLM Compression?

A naive approach would ask a strong LLM to read the full context and summarize or select atoms before making a final call. That fails the cost goal. If the compressor already sends the full context to a large LLM, the compression system has paid most of the cost before the actual answer is generated. Therefore, the deployed compressor must avoid a large semantic model in the hot path.

The acceptable uses of large models are:

- offline generation of labels;
- offline error analysis and evaluation;
- optional expensive audit mode for high-risk enterprise workflows;
- synthetic stress-test creation.

The default online path uses retrieval features, an encoder reranker or distilled scorer, a deterministic budget allocator, and cheap risk checks.

3 High-Level Architecture

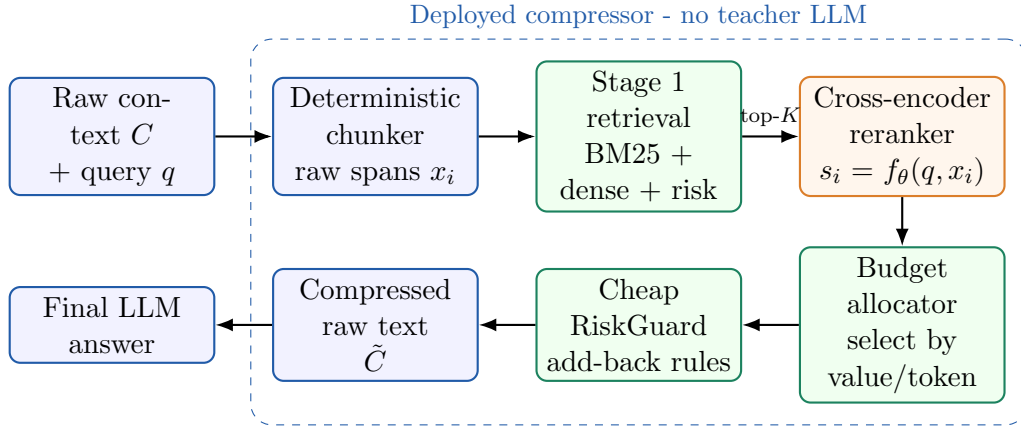


Figure 1: Online inference architecture. The system consumes raw text and outputs raw text. Embeddings and model activations are internal scoring tools only; they are never sent to the final LLM.

3.1 Main Components

1. **Chunker.** Splits raw text into safe contiguous spans while preserving semantic units such as sentences, bullet items, table rows, code blocks, and policy clauses.
2. **Stage-1 candidate generator.** Uses BM25, dense embedding similarity, and simple risk features to retrieve high-recall top- K candidate chunks.
3. **Cross-encoder evidence reranker.** Scores query-chunk pairs using joint attention over $[q; x_i]$, producing an evidence utility score s_i .
4. **Budget allocator.** Selects chunks under a hard token budget using score-per-token with redundancy control.
5. **RiskGuard.** Runs cheap deterministic checks for high-risk dropped spans, especially numbers, negations, code identifiers, dates, exceptions, table rows, and exact constraints.
6. **Distilled student.** Optional cheap production model trained to approximate the cross-encoder score from lightweight features.

4 Raw-Text Chunking

The chunker is deployed as a deterministic preprocessing module inside the application backend. It does not require a GPU and does not call an LLM. Its purpose is to create candidate spans that are large enough to preserve meaning but small enough for selective compression.

4.1 Chunking Rules

A robust chunker follows this boundary priority:

1. preserve code blocks, tables, bullet lists, numbered requirements, and Markdown sections;
2. split by headings and paragraphs;
3. split long paragraphs into sentence groups;
4. use token-window fallback only when structure is unavailable;
5. add controlled overlap, usually 20-40 tokens.

Target chunk size for a hackathon MVP: 120-250 tokens. For code and tables, chunk by syntactic or row-level structure rather than fixed tokens.

4.2 Chunking Failure Modes

Chunking can destroy meaning before the model sees the data. The most dangerous failure is splitting a constraint from its qualifier:

Bad split	Safe split
Chunk 1: Students may submit late. Chunk 2: Unless the assignment is the final essay.	Chunk 1: Students may submit late unless the assignment is the final essay.

The system should avoid splitting around words such as **not**, **unless**, **except**, **only if**, **before**, **after**, **must**, and **cannot**. For code, function signatures and their immediate docstrings should remain together. For tables, each row should remain intact.

5 Stage-1 Candidate Generation

The cross-encoder is too expensive to run over every chunk for very long contexts. Therefore, the first stage retrieves a high-recall candidate set $\mathcal{K} \subset \{1, \dots, n\}$.

5.1 Hybrid Candidate Score

For each chunk x_i :

$$r_i = \alpha \text{BM25}(q, x_i) + \beta \cos(e_q, e_i) + \gamma h_i, \quad (4)$$

where e_q and e_i are query and chunk embeddings, and h_i is a cheap risk prior. The risk prior is high for chunks containing exact values and fragile semantics:

- numbers, dates, prices, percentages, units;
- negation and exception words;
- code identifiers and function names;
- table rows and legal/policy constraints;
- headings matching query entities.

The top- K chunks under r_i are passed to the cross-encoder. Typical values: $K = 64$ for low latency, $K = 128$ for quality, and $K = 256$ for offline evaluation.

6 Cross-Encoder Evidence Reranker

The main quality model is a cross-encoder reranker. Unlike a bi-encoder, which embeds the query and chunk separately, a cross-encoder jointly attends over the query and chunk. This allows it to model interactions such as negation, directionality, exception clauses, and entity-role matching.

6.1 Model Form

For each candidate chunk x_i :

$$s_i = f_{\theta}(q, x_i) = w^{\top} \text{Enc}_{\theta}([\text{CLS}], q, [\text{SEP}], x_i)_{\text{CLS}} + b. \quad (5)$$

The score s_i estimates whether x_i is useful evidence for answering q . A ModernBERT-style encoder is a strong backbone because encoder-only models are efficient for classification and reranking tasks, and the pairwise input fits the reranker setting.

6.2 Why This Is the Right Model Layer

The problem is not answer generation. The problem is evidence selection. A generative LLM fine-tune would waste time and compute. A cross-encoder directly trains the function we need:

Given a query and a candidate text chunk, assign a high score if the chunk is needed for the answer.

This is aligned with supporting-fact datasets and can be trained quickly with supervised labels.

7 Training Data Construction

7.1 Primary Dataset: HotpotQA

HotpotQA provides questions, Wikipedia context paragraphs, answers, and sentence-level supporting facts. It is ideal for training a chunk selector because supporting facts can be converted directly into positive chunks.

For each example:

$$q = \text{question}, \quad (6)$$

$$X = \{x_1, \dots, x_n\} \quad \text{sentences or sentence groups}, \quad (7)$$

$$y_i = \mathbb{I}[x_i \text{ contains a gold supporting fact}]. \quad (8)$$

7.2 Secondary Dataset: QASPER

QASPER contains questions over NLP papers and annotated supporting evidence. It is useful because scientific papers are longer and closer to real document QA. The evidence annotations can label positive chunks at paragraph or section level.

7.3 Hard Negative Mining

The model must learn to reject chunks that are semantically similar but not actually necessary. For each positive chunk, construct negatives:

- **BM25 hard negatives:** high lexical overlap but not supporting evidence;
- **dense hard negatives:** high embedding similarity but not gold evidence;
- **entity-confuser negatives:** share names/numbers but express a different relation;
- **random negatives:** low relevance, used for calibration.

A good ratio is 1 positive : 4 hard negatives : 4 random negatives.

8 Training Objective

8.1 Binary Evidence Loss

The base loss is binary cross entropy over candidate chunks:

$$\mathcal{L}_{\text{BCE}} = - \sum_i [y_i \log \sigma(s_i) + (1 - y_i) \log(1 - \sigma(s_i))] . \quad (9)$$

This trains the model to score evidence chunks high and non-evidence chunks low.

8.2 Pairwise Ranking Loss

For each positive chunk x^+ and negative chunk x^- from the same query:

$$\mathcal{L}_{\text{rank}} = \sum_{(x^+, x^-)} \max(0, m - s^+ + s^-), \quad (10)$$

where m is a margin. This directly teaches the model to rank true evidence above distractors.

8.3 Risk-Weighted Loss

Some chunks contain fragile facts: negations, numbers, code identifiers, dates, or policy exceptions. Missing these can cause silent critical loss. Assign a risk weight ρ_i :

$$\mathcal{L}_{\text{risk}} = - \sum_i \rho_i [y_i \log \sigma(s_i) + (1 - y_i) \log(1 - \sigma(s_i))] . \quad (11)$$

A practical training objective is:

$$\mathcal{L} = \mathcal{L}_{\text{BCE}} + \lambda \mathcal{L}_{\text{rank}} + \mu \mathcal{L}_{\text{risk}} . \quad (12)$$

9 Budgeted Selection

After reranking, the compressor selects raw chunks under a token budget B . Let t_i be the token count of chunk i . Let s_i be the reranker score. Let $R(S)$ be a redundancy penalty for selected set S .

9.1 Optimization Target

The ideal problem is:

$$\max_{S \subseteq \mathcal{K}} \sum_{i \in S} u_i - \eta R(S) \quad (13)$$

$$\text{s.t.} \quad \sum_{i \in S} t_i \leq B, \quad (14)$$

where u_i is a calibrated utility derived from s_i and risk features.

9.2 Greedy Approximation

In the MVP, use a greedy value-per-token rule:

$$\text{priority}_i = \frac{u_i}{t_i^\alpha}, \quad \alpha \in [0.8, 1.0]. \quad (15)$$

After selecting a chunk, reduce the utility of near-duplicate chunks:

$$u_j \leftarrow u_j - \eta \max_{i \in S} \cos(e_i, e_j). \quad (16)$$

This is fast, stable, and easy to explain.

10 RiskGuard: Cheap Add-Back Without LLM Verification

RISKGUARD is not an LLM verifier. It is a deterministic safety layer that runs after the budget allocator. Its job is to prevent obvious high-risk omissions using cheap checks.

10.1 Risk Rules

A dropped chunk x_j receives an add-back candidate flag if:

$$\text{risk}(x_j) > \tau_r \quad \text{and} \quad \text{rel}(q, x_j) > \tau_q. \quad (17)$$

Risk triggers include:

- exact numbers or units;
- dates, times, deadlines, prices;
- negation and exception words;
- function names, class names, API signatures;

- table rows where query terms match a column or row header;
- policy/legal words such as **must**, **cannot**, **required**, **unless**.

If budget remains, RISKGUARD adds the highest-risk dropped chunks. If budget is exhausted, it can swap out lower-value generic chunks.

11 Distilled Cheap Student

The cross-encoder gives best quality but may be too expensive for every production call. Therefore, train a cheap student to approximate cross-encoder scores.

11.1 Teacher Scores

Run the trained cross-encoder over many query-chunk pairs and save:

$$z_i = f_\theta(q, x_i). \quad (18)$$

11.2 Student Features

The student consumes cheap features:

- BM25 score;
- dense cosine similarity;
- query term overlap;
- entity, number, and date overlap;
- negation/exception indicators;
- code identifier overlap;
- token count;
- chunk position;
- section-heading match;
- redundancy features.

11.3 Student Objective

For a regression student h_ϕ :

$$\mathcal{L}_{\text{distill}} = \sum_i (h_\phi(\psi(q, x_i)) - z_i)^2. \quad (19)$$

For a ranking student, use pairwise or LambdaRank-style objectives. LightGBM is a strong default because it trains fast and handles heterogeneous features well.

12 Inference Modes

Mode	Scoring model	Cost	Use case
Quality mode	Cross-encoder reranker	Medium	Demos, high-value requests, batch compression
Cheap mode	Distilled student	Low	Production default, low-latency API calls
Fallback mode	BM25 + embeddings + risk rules	Very low	If training fails or model unavailable
Audit mode	Optional offline LLM verifier	High	Evaluation, regulated workflows, not default inference

Table 1: Deployment modes. The online default never requires a teacher LLM or full-context semantic verifier.

13 Computational Complexity

Let n be the number of chunks and K the number of candidate chunks passed to the cross-encoder.

- Chunking: $O(|C|)$.
- BM25 retrieval: approximately $O(n)$ per query with an inverted index, often faster in practice.
- Dense scoring: $O(nd)$ if embeddings are computed online, lower if chunk embeddings are cached.
- Cross-encoder reranking: $O(KL^2)$ for sequence length L , depending on backbone attention implementation.
- Greedy selection: $O(K \log K)$ plus redundancy checks.

The key latency control knob is K . In many deployments, chunk embeddings can be cached, making online cost mostly query embedding, sparse retrieval, top- K reranking, and prompt assembly.

14 Evaluation Plan

14.1 Evidence Retrieval Metrics

For datasets with gold evidence:

$$\text{EvidenceRecall@B} = \frac{|E_{\text{gold}} \cap E_{\text{selected}}|}{|E_{\text{gold}}|}. \quad (20)$$

Report evidence recall at compression budgets of 25%, 50%, and 75%.

14.2 Answer Quality Metrics

For QA tasks, run the final LLM over compressed context and compare to gold answers using exact match and token F1. For summarization tasks, use ROUGE/BERTScore or a structured rubric. For the hackathon, answer F1 and qualitative failure examples are enough.

14.3 Compression Metrics

$$\text{CompressionRatio} = \frac{T(C)}{T(\tilde{C})}. \quad (21)$$

Also report final token cost saved and latency.

14.4 Critical-Loss Stress Tests

Create synthetic contexts where the answer depends on a small fragile span:

- a negation hidden in the middle;
- a date or deadline that changes eligibility;
- a table row with the correct room/time;
- a code signature where one parameter matters;
- a policy exception clause.

The metric is silent critical loss: the fraction of examples where the compressed answer is confident but violates the fragile fact.

15 Baselines

Compare against:

1. **Raw truncation:** keep the first B tokens.
2. **BM25 top-k:** select chunks by lexical relevance.
3. **Dense top-k:** select by embedding similarity.
4. **Hybrid retrieval only:** BM25 + dense + risk features without cross-encoder reranking.
5. **Summarization baseline:** use a cheap summarizer if allowed, but note runtime cost.
6. **LLMLingua-style baseline:** if setup is stable within time.

The most important win condition is not just answer F1. It is demonstrating that the cross-encoder preserves evidence that retrieval-only baselines drop.

16 Ablation Studies

Ablation	Question answered
Remove cross-encoder	How much does joint query-chunk attention help over retrieval?
Remove dense retrieval	Is lexical retrieval sufficient?
Remove BM25	Are embeddings alone enough?
Remove RiskGuard	Does deterministic add-back reduce critical loss?
Remove hard negatives	Does model fail on semantically similar distractors?
Use sentence chunks vs paragraph chunks	Which chunk granularity preserves evidence better?
Distilled student vs cross-encoder	How much quality is lost for cheaper inference?

Table 2: Recommended ablations for an 8-hour build.

17 Failure Modes and Mitigations

17.1 Stage-1 Retrieval Misses Gold Evidence

If the gold evidence is not in top- K , the cross-encoder cannot recover it. Mitigation: increase K , include BM25 and dense retrieval, add risk-triggered candidates, and preserve section headings.

17.2 Chunk Boundary Corrupts Meaning

If a qualifier is split away from a claim, the model may select the wrong half. Mitigation: semantic boundary rules, overlap, and special handling for negation/exception clauses.

17.3 Cross-Encoder Overfits HotpotQA

HotpotQA is Wikipedia-style. It may not transfer to code, legal, or scientific docs. Mitigation: mix QASPER, synthetic policy/code examples, and domain-specific fine-tuning.

17.4 Reranker Scores Are Not Token-Aware

A high-scoring long chunk may be less cost-effective than several shorter chunks. Mitigation: use score-per-token and length-calibrated utility.

17.5 RiskGuard Adds Too Much

Conservative add-back can reduce compression ratio. Mitigation: cap risk add-back budget, require moderate query relevance, and allow swapping rather than only adding.

17.6 Distilled Student Loses Semantics

A cheap student may miss relational details. Mitigation: keep quality mode available, distill on hard negatives, include interaction features, and evaluate student gap explicitly.

18 Decision Junctions

Decision	Fast choice	Higher-quality choice
Backbone	ModernBERT-base	ModernBERT-large
Candidate K	64	128-256
Chunking	sentence groups	structure-aware document parser
Training loss	BCE	BCE + pairwise ranking + risk weights
Dataset	HotpotQA only	HotpotQA + QASPER + synthetic critical-loss
Deployment model	distilled student	cross-encoder quality mode
Budget selection	score/token greedy	redundancy-aware submodular greedy

19 Eight-Hour Build Plan

1. **Hour 0-1: Data pipeline.** Load HotpotQA, convert supporting facts into chunk labels, build positive and negative pairs.
2. **Hour 1-2: Baselines.** Implement chunking, BM25, dense retrieval, token counting, and raw truncation.
3. **Hour 2-4: Train cross-encoder.** Start ModernBERT-base sanity run; launch ModernBERT-large in parallel if compute allows.
4. **Hour 4-5: Integrate compression.** Candidate generation, reranking, budget selection, prompt assembly.
5. **Hour 5-6: Distill student.** Generate teacher scores for many pairs; train LightGBM/MLP student.
6. **Hour 6-7: Evaluate.** Evidence recall, answer F1, compression ratio, critical-loss stress tests.
7. **Hour 7-8: Demo.** Show side-by-side failures of truncation/retrieval and wins by reranking.

20 Implementation Skeleton

```
def compress(context: str, query: str, budget_tokens: int, mode="quality") -> str:
    chunks = chunk_text(context)
    token_counts = count_tokens(chunks)

    # Stage 1: high-recall candidate generation
    bm25_scores = bm25_score(query, chunks)
    dense_scores = dense_similarity(query, chunks)
    risk_scores = risk_prior(chunks)
    retrieval_scores = 0.45 * bm25_scores + 0.45 * dense_scores + 0.10 * risk_scores
    candidates = top_k(chunks, retrieval_scores, k=128)

    # Stage 2: rerank or use distilled student
    if mode == "quality":
        scores = cross_encoder_score(query, candidates)
```

```

elif mode == "cheap":
    feats = featurize(query, candidates)
    scores = student_score(feats)
else:
    scores = retrieval_scores[candidates]

selected = greedy_budget_select(candidates, scores, token_counts, budget_tokens)
selected = risk_add_back(selected, chunks, query, budget_tokens)
return assemble_prompt(query, selected)

```

21 Startup Positioning

The pitch should be simple:

ATOMCODEC-RERANK is a deployable prompt compressor that learns to preserve evidence, not just semantic similarity. It trains a query-conditioned reranker on supporting-fact supervision and uses it to select the smallest raw-text evidence set under a token budget. The production path does not call a teacher LLM, does not extract atoms at runtime, and does not require a semantic verifier. It is a model-light compression layer that can sit in front of any LLM API.

The strongest demo is a cost-quality curve: same or better answer accuracy than retrieval baselines at lower token budget, plus examples where the system preserves small decisive details that naive compression drops.

22 Conclusion

ATOMCODEC-RERANK reframes prompt compression as query-conditioned evidence selection. The core move is to train the model that is actually needed: a reranker that knows whether a chunk is useful evidence for a query. The architecture avoids the fatal product flaw of runtime teacher-LLM compression. It remains raw-text, model-agnostic, deployable, and measurable.

In eight hours with strong GPUs, the right bet is not a generative LLM fine-tune. It is a supervised cross-encoder evidence reranker, a reliable retrieval-rerank-selection pipeline, and a distilled cheap student for the production story.

References

- [1] Zhilin Yang et al. *HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering*. EMNLP, 2018.
- [2] Pradeep Dasigi et al. *A Dataset of Information-Seeking Questions and Answers Anchored in Research Papers*. NAACL, 2021.
- [3] Yushi Bai et al. *LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding*. ACL, 2024.
- [4] Benjamin Warner et al. *Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference*. arXiv, 2024.
- [5] Nils Reimers and Iryna Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP-IJCNLP, 2019.